

GrapHist: Complete Hyperparameter Reference

Every parameter in the paper and in the released code

Group 1 · Aadith Krishna, Aditya Santosh, Arjungopal Anilkumar

How to read this document. Values marked **required** have no default and must be supplied on the command line. Values marked **hardcoded** are fixed constants inside the source and cannot be changed without editing the code. Values marked **dead** are parsed and passed through the call chain but never reach the model; they are listed in Section 8 with the evidence. Where the paper and the code disagree, both are given.

Contents

1	Data preprocessing pipeline	2
1.1	Stage 00: tiling (paper only, no script released)	2
1.2	Stage 01: cell segmentation	2
1.3	Stage 02: cell feature extraction	2
1.4	Stage 03: graph construction	3
1.5	Stage 04: serialise to .pt	3
2	Model architecture	4
2.1	ACM-GIN block	4
2.2	Encoder, decoder and expressivity additions	4
2.3	Transform pipeline (applied in this exact order)	4
3	Self-supervised pre-training	6
4	Embedding generation	8
5	Cell-level evaluation	9
6	Region- and slide-level evaluation	11
6.1	Region level	11
6.2	Slide level and MIL	11
7	The paper's search ranges and baseline settings	13
7.1	Self-supervised pre-training (paper Section 4.2)	13
7.2	Self-supervised evaluation (paper Appendix C)	13
7.3	Supervised baselines (paper Appendix C)	13
7.4	Model sizes (paper Table C.7)	13
8	Parameters declared but never used	14
9	Where the paper and the code disagree	14
10	Our environment and the settings we changed	14

1 Data preprocessing pipeline

The four scripts in `src/data/` convert whole slide images into the `.pt` cell graphs. Stages 01–04 use the `fire` CLI, so arguments are positional-or-keyword Python parameters rather than `argparse` flags.

1.1 Stage 00: tiling (paper only, no script released)

Parameter	Value	Note
Tissue segmentation	per paper ref. [43]	separates tissue from slide background
Magnification	20×	standard digital pathology convention
Resolution	0.5 μm /pixel	matches <code>img_mpp</code> in stage 02
Tile size	224 × 224 px	
Tile overlap	none	non-overlapping
Corpus	998 WSIs 11,149,500 tiles	→ TCGA-BRCA

1.2 Stage 01: cell segmentation

`src/data/01_cell_segmentation.py`

Parameter	Value	Note
<code>input_path</code>	required	directory of patient folders of patch images
<code>output_path</code>	required	
Model	StarDist2D	nuclei instance segmentation, U-Net backbone
Pretrained weights	2D_versatile_he	H&E-specific
<code>scale</code>	1	hardcoded , tuned for 20× input
<code>nms_thresh</code>	0.01	hardcoded , non-maximum suppression
<code>prob_thresh</code>	0.25	hardcoded , detection probability floor
Device	GPU	0.127 s/patch (paper Table C.8)

1.3 Stage 02: cell feature extraction

`src/data/02_cell_feature_extraction.py`

Parameter	Value	Note
<code>input_path</code>	required	patch images
<code>seg_path</code>	required	stage 01 output
<code>output_path</code>	required	
<code>n_jobs</code>	24	CPU parallelism
<code>img_mpp</code>	0.50	hardcoded microns per pixel; fixes the physical scale
GLCM distances	[1]	hardcoded co-occurrence offset
GLCM angles	$0, \pi/4, \pi/2, 3\pi/4$	hardcoded four directions
GLCM levels	256	hardcoded
GLCM symmetric	True	hardcoded
GLCM normed	True	hardcoded
Fourier n	[20, 30]	hardcoded two rfft lengths
Fourier workers	3	hardcoded
Device	CPU	0.369 s/patch (paper Table C.8)

The 96 node features, decomposed. Verified by tracing the extraction code rather than inferring from the paper’s Table B.6.

Group	Count	Composition
Colour intensity	24	min, max, mean, std, skew, kurtosis over R, G, B (18) plus the same six over greyscale (6)
Morphology	11	detection probability, orientation, major axis length, minor axis length, eccentricity, area, perimeter, circularity, elongation, solidity, extent
GLCM texture	36	6 properties (ASM, contrast, correlation, dissimilarity, energy, homogeneity) $\times$ 6 statistics (mean, std, skew, kurtosis, min, max)
Fourier shape	25	<code>rrfft</code> of the centroid-distance signature at $n = 20$ and $n = 30$ gives $20/2 + 1 = 11$ and $30/2 + 1 = 16$ coefficients; each is divided by its DC term, which is then dropped, leaving $10 + 15$
Total	96	matches the paper's stated node feature dimension

1.4 Stage 03: graph construction

`src/data/03_graph_construction.py`

Parameter	Value	Note
<code>feat_path</code>	required	stage 02 output
<code>output_path</code>	required	
<code>dist_threshold</code>	100	microns; edges longer than this are removed
Topology	Delaunay triangulation	over <code>center_X</code> , <code>center_Y</code>
Edge weight	Euclidean distance	in microns, range (0, 100)
Edge storage	one-directional	every <code>edge_index</code> has <code>src < dst</code>
Device	CPU	0.009 s/patch (paper Table C.8)

1.5 Stage 04: serialise to .pt

`src/data/04_save_dataset.py`

Parameter	Value	Note
<code>edge_path</code>	required	stage 03 output
<code>clinical_data_path</code>	required	labels and metadata
<code>output_path</code>	required	one <code>.pt</code> per tile

2 Model architecture

2.1 ACM-GIN block

Parameter	Value	Note
Random-walk matrix	$A_{rw} = D^{-1}A$	D is the <i>weighted</i> degree matrix
Low-pass channel	$A_L = (I + A_{rw})/2$	
High-pass channel	$A_H = (I - A_{rw})/2$	
Identity channel	$A_I = I$	
Temperature T	3.0	hardcoded default in <code>ACM_GIN.__init__</code>
Aggregation	<code>aggr="add"</code>	hardcoded in <code>kwargs.setdefault</code>
Message function	<code>edge_weight * x_j</code>	
Degree normalisation	<code>1 / weighted_degree</code>	guarded by <code>masked_fill_(deg_inv == inf, 0)</code>
Channel importance	<code>sigmoid(Linear(d, 1))</code>	one per channel
Mixing matrix	<code>Linear(3, 3)</code>	applied to the three scores divided by T
Final combination	<code>softmax, convex sum</code>	per node, over the three channels
Per-channel MLP	<code>Linear(in, hid) → act →</code> <code>Linear(hid, out) → act</code>	<code>BatchNorm1d</code> inserted after each <code>Linear</code> only if <code>batchnorm=True</code>

2.2 Encoder, decoder and expressivity additions

Component	Value	Note
Encoder type	<code>acm_gin</code>	
Encoder depth	5	paper: chosen following Hou et al. for large datasets
Decoder type	<code>acm_gin</code>	
Decoder depth	1	hardcoded as <code>num_layers=1</code> in <code>PreModel</code>
Embedding dim d	512	released checkpoint; searched over {512, 768, 1024}
Mask token	<code>nn.Parameter(zeros(1, in_dim))</code>	learnable
Jumping knowledge	<code>concat_hidden=True</code>	concatenates all layer outputs
Encoder→decoder	<code>Linear(d×num_layers, d, bias=False)</code>	with JK; <code>Linear(d, d, bias=False)</code> without
Virtual node features	<code>zeros</code>	
Virtual node edge weight	22.5	hardcoded . Paper states “the mean of all edge weights in the pre-training dataset”. See Section 9.
Parameter count	9.50M / 21.12M / 37.34M	at $d = 512 / 768 / 1024$

2.3 Transform pipeline (applied in this exact order)

Transform	Parameter	Note
<code>ToUndirected()</code>	—	released edges are stored one-directional
<code>NormalizeData()</code>	<code>normalization.json</code>	z-scores node features <i>and</i> edge weights
<code>AddVirtualNode()</code>	weight 22.5	inserted <i>after</i> normalisation

Contents of `normalization.json`.

Key	Value	Note
edge_attr.mean	18.62691879	microns, over the pre-training corpus
edge_attr.std	12.73480892	microns
x.mean	96-vector	one entry per node feature
x.std	96-vector	

3 Self-supervised pre-training

src/train/pretrain.py

Flag	Default	Note
<i>General</i>		
-dataset	TCGA_BRCA	
-seed	0	
<i>Paths</i>		
-save_folder	required	
-norm_json_path	required	
-metadata_csv_path	required	
<i>Model</i>		
-encoder	acm_gin	only acm_gin is implemented
-decoder	acm_gin	
-num_hidden	required	embedding dimension d
-num_layers	required	encoder depth
-mask_rate	required	paper searched {0.50, 0.75}
-replace_rate	required	paper searched {0.00, 0.10}
-drop_edge_rate	0.0	
-node_pooling	mean	
-activation	prelu	
-batchnorm	False	
-concat_hidden	True	jumping knowledge
-loss_fn	sce	scaled cosine error
-alpha_l	3	the exponent γ in the SCE loss
-num_heads	4	dead architecturally; only asserts num_hidden % nhead == 0
-num_out_heads	1	dead architecturally; same assertion
-residual	None	dead
-attn_drop	0.1	dead
-in_drop	0.2	dead
-norm	None	dead
-negative_slope	0.2	dead
-dropout	0.2	dead
<i>Optimisation</i>		
-optimizer	adam	
-lr	0.001	
-weight_decay	0	
-scheduler	False	no learning-rate schedule by default
-max_epoch	100	same budget as the paper's vision baselines
-batch_size	2048	same as the vision baselines
-num_workers	4	
<i>Run control</i>		
-train	True	
-load_model	False	
-resume_run	False	
-save_model	True	
-logging	True	
<i>Inferred at runtime, not flags</i>		
num_features	96	read from train_dataset[0].num_features

Flag	Default	Note
num_edge_features	1	read from <code>train_dataset[0].edge_attr.size(1)</code>

Masking procedure. A fraction `mask_rate` of nodes is selected. Of those, a fraction `replace_rate` has its features replaced by another node’s features; the remaining `1 - replace_rate` receives the learnable mask token. The virtual node is excluded from masking.

Loss. Scaled cosine error over masked nodes only:

$$L_{SCE} = \frac{1}{|\tilde{V}|} \sum_{v_i \in \tilde{V}} \left(1 - \frac{x_i^\top h_i}{\|x_i\| \cdot \|h_i\|} \right)^\gamma, \quad \gamma = 3.$$

4 Embedding generation

src/train/generate_embs.py

Model flags must match the checkpoint. Note that where `pretrain.py` marks a flag **required**, this script supplies a concrete default matching the released checkpoint.

Flag	Default	Note
-dataset	required	
-scale	required	one of <code>cell</code> , <code>patch</code> , <code>slide</code>
-model_path	required	<code>weights/graphist.pt</code>
-norm_json_path	required	
-metadata_csv_path	None	required for <code>patch</code> and <code>slide</code> scale
-cell_graphs_path	None	required for <code>cell</code> scale
-output_path	required	
-num_hidden	512	matches the released checkpoint
-num_layers	5	matches the released checkpoint
-encoder / -decoder	<code>acm_gin</code>	
-mask_rate	0.5	unused at inference
-replace_rate	0.1	unused at inference
-drop_edge_rate	0.0	
-node_pooling	<code>mean</code>	
-num_edge_features	1	
-num_heads	4	dead
-num_out_heads	1	dead
-residual	None	dead
-attn_drop	0.1	dead
-in_drop	0.2	dead
-norm	None	dead
-negative_slope	0.2	dead
-batchnorm	<code>False</code>	
-activation	<code>prelu</code>	
-loss_fn	<code>sce</code>	
-alpha_l	3	
-concat_hidden	<code>True</code>	
-max_epoch	100	unused at inference
-batch_size	2048	we used 8 for BRACS, 64 for BreakHis
-num_workers	4	we used 0 (see Section 10)
-lr	0.001	unused at inference
-seed	0	

Embedding construction by scale.

- **Cell:** the node embedding, with the virtual node excluded.
- **Patch / region:** the mean over nodes. *Includes* the virtual node, unlike the cell path.
- **Slide:** the mean over nodes per patch, then attention MIL across patches. Also includes the virtual node.

5 Cell-level evaluation

src/evaluate/main_cell.py

Flag	Default	Note
-split_mode	required	one of <code>csv</code> , <code>folder</code> , <code>group</code>
-emb_dir	None	single directory, for <code>group</code> mode
-fold_dirs	None	one directory per fold, for <code>folder</code> mode
-split_csv_dir	None	fold CSVs, for <code>csv</code> mode
-num_folds	None	auto-detected in <code>csv</code> mode; defaults to 5 in <code>group</code> mode
-group_regex	None	first capture group becomes the grouping key
-tissue_filter	None	e.g. <code>Breast</code> for the PanNuke breast subset
-label_map	None	e.g. <code>nucls_super</code> for the 7→4 mapping
-save_dir	required	
-seed	0	
-C	1.0	inverse regularisation strength
-max_iter	2000	lbfgs iteration cap
-class_weight	balanced	re-weights classes inversely to frequency

Probe configuration. A scikit-learn Pipeline:

Step	Setting	Note
StandardScaler	defaults	zero mean, unit variance per dimension
LogisticRegression solver	lbfgs	multinomial is the lbfgs default
C	1.0	
max_iter	2000	
class_weight	balanced	
random_state	0	

Splits actually used.

Dataset	Mode	Configuration
PanNuke	folder	the dataset's own three published folds, <code>fold_1..3</code>
PanNuke Breast	folder	same three folds, with <code>-tissue_filter Breast</code>
NuCLS	group	<code>GroupKFold(n_splits=5),</code> <code>'^(TCGA-[A-Z0-9]+)'</code> <code>-group_regex</code>

Why 5 folds for NuCLS. The NuCLS dataset card documents a `splits/` directory of five train and test fold CSVs, numbered `fold_1` to `fold_5`, each carrying a `hospital` column. The dataset's own protocol is therefore **5 hospital-based folds**. That directory is absent from the published repository, so the exact assignment cannot be recovered. We therefore matched the fold *count* (5) and the grouping *principle* (hospital) by grouping on the TCGA tissue source site, which for TCGA barcodes is the contributing institution. This yields **18 groups** over 1,694 graphs.

Fold	Train	Test	Sites	Sites held out
1	1,355	339	2	A2, GI
2	1,358	336	3	BH, E2, LL
3	1,357	337	4	AN, AO, AQ, D8
4	1,353	341	5	A1, C8, EW, OL, S3
5	1,353	341	4	A7, AC, AR, GM

Note that **GroupKFold** is *not* stratified, so rare classes can disappear from a fold entirely. This is what happens to the NuCLS “Other” class (587 cells) and is why the released AUROC computation had to be made degeneracy-tolerant.

6 Region- and slide-level evaluation

6.1 Region level

src/evaluate/main_patch.py

Flag	Default	Note
-dataset	required	
-seed	0	
-metadata_csv	required	replaced a PATH/T0/... placeholder
-embs_path	required	directory of {sample_id}.npz
-save_dir	required	

6.2 Slide level and MIL

src/evaluate/main_slide.py

Flag	Default	Note
-dataset	required	
-mode	required	
-classifier	attentive	one of attentive, additive, conjunctive
-input_dim	512	must match the embedding dimension
-mask_rate	0.50	
-replace_rate	0.10	
-label_path	required	
-train_data_path	required	
-test_data_path	required	
-project_dir	required	
-save_dir	required	
-seed	0	
-epochs	200	
-batch_size	64	
-n_splits	5	stratified cross-validation on the training set
-patience	15	early stopping
-delta	0.01	early stopping minimum improvement

Training configuration, fixed in code.

Component	Setting	Note
Loss	nn.CrossEntropyLoss()	on bag_logits
Optimiser	optim.Adam	lr from the grid
Scheduler	StepLR	step_size = epochs / 10, gamma = 0.5
Early stopping	EarlyStopping	patience=15, delta=0.01
Model selection	highest mean validation macro F1	across the 5 CV folds

MIL hyperparameter grid, as implemented.

Parameter	Search space (code)	Paper's stated range
attn_hidden_dim	[128, 256]	{128, 256}
clf_hidden_dim	[[], [input_dim]]	1 or 2 classifier layers
dropout	[0.2, 0.5]	{0.2, 0.5}
lr	[0.001, 0.01]	{0.001, 0.01}

Parameter	Search space (code)	Paper's stated range
Aggregators	3 variants	ABMIL, additive, conjunctive
Total per dataset	16 configurations $\times$ 3 aggregators	

Redundancy. Dropout is inserted only when the classifier has a hidden layer, and the attention module has none. With `clf_hidden_dim=[]` the two dropout values therefore produce identical runs, so **4 of the 16 configurations duplicate**, per dataset, per aggregator.

MIL module defaults. `attentive_mil.py`, `additive_mil.py`, `conjunctive_mil.py`: `hidden_dim=()`, `hidden_activation=nn.ReLU()`, `dropout=0.0`, `attentive_function=Sum()`.

7 The paper’s search ranges and baseline settings

7.1 Self-supervised pre-training (paper Section 4.2)

Parameter	Range searched	Note
Embedding dimension d	{512, 768, 1024}	validated following Hou et al.
Masking ratio r_m	{0.50, 0.75}	
Replacement ratio r_r	{0.00, 0.10}	
Encoder depth	5	fixed, not searched
Decoder depth	1	fixed, not searched
Epochs	100	identical for GrapHist and the vision baselines
Batch size	2048	identical for GrapHist and the vision baselines
Hardware	A100 80GB, H200 140GB	

7.2 Self-supervised evaluation (paper Appendix C)

For slide- and region-level tasks, a 5-fold stratified cross-validation on the training set selects the MIL head hyperparameters; the configuration with the highest mean validation macro F1 is then evaluated on the held-out test set. The procedure is repeated for all three aggregators and the best variant reported. For cell-level tasks the datasets’ own train/test splits are used with logistic regression.

7.3 Supervised baselines (paper Appendix C)

Task level	GNN hidden	GNN layers	Head hidden	Cross-validation
Slide / region	{16, 32}	{1, 2}	{64, 128} attention	5-fold stratified
Cell	{256, 512}	{3, 5}	{64, 128} MLP	2-fold PanNuke, 4-fold NuCLS

Neither baseline has training code in the released repository, so these ranges could not be verified against an implementation.

7.4 Model sizes (paper Table C.7)

Model	Params (10^6)	Note
Vision baseline 1	22.01	ViT-S/16 backbone
Vision baseline 2	47.58	ViT-S/16 backbone
GrapHist ($d = 512$)	9.50	the released checkpoint
GrapHist ($d = 768$)	21.12	
GrapHist ($d = 1024$)	37.34	

8 Parameters declared but never used

Seven hyperparameters are parsed by `argparse`, read by `build_model`, and accepted by `PreModel.__init__`, but never reach the network. The evidence is `setup_module`, which is the only function that constructs an `ACM_GIN_model` and accepts exactly seven arguments:

```
def setup_module(m_type, in_dim, out_dim, num_hidden,
                 num_layers, activation, batchnorm) -> nn.Module:
    if m_type == "acm_gin":
        mod = ACM_GIN_model(int(in_dim), int(out_dim), num_layers,
                             int(num_hidden), batchnorm, activation=activation)
```

Flag	Default	Consequence
<code>-residual</code>	None	no residual connections are configurable
<code>-norm</code>	None	<code>create_norm</code> and <code>NormLayer</code> are dead code
<code>-negative_slope</code>	0.2	no effect on any activation
<code>-attn_drop</code>	0.1	no attention dropout
<code>-in_drop</code>	0.2	no input dropout, despite the README suggesting <code>-in_drop 0.2</code>
<code>-dropout</code>	0.2	no dropout anywhere
<code>-num_heads</code>	4	only used in <code>assert num_hidden % nhead == 0</code>

Net effect. Since `-batchnorm` also defaults to `False`, GrapHist pre-trains with **no dropout and no normalisation layers of any kind**.

9 Where the paper and the code disagree

Item	Paper	Code
Virtual node edge weight	“set to the mean of all edge weights in the pre-training dataset”	hardcoded 22.5, inserted <i>after</i> z-scoring. The correctly scaled value is $(22.5 - 18.627)/12.735 = 0.304$, so the injected value is about $74\times$ too large
Probing	described as “non-linear probing” three times	plain <code>LogisticRegression</code> ; it is a linear probe
Patch embedding	“the mean of its cell embeddings”	means over <i>all</i> nodes, including the virtual node
MIL normalisation	implied	computed, written to <code>normalization.json</code> , then discarded; <code>__getitem__</code> returns the raw tensor

Measured effect of the virtual node weight , on 300 PanNuke graphs:

Configuration	$\text{deg} \leq 0$	$\ \text{neigh}\ /\ \text{self}\ $	$\cos(\text{low}, \text{high})$
As shipped, 22.5	0.0%	0.080	+0.992
Naive correction, 0.304	57.8%	1.009	−0.012
Gaussian kernel, $\sigma = 25\mu\text{m}$	0.0%	0.787	+0.320
Inverse $1/(1 + d/\mu)$	0.0%	0.785	+0.325
Linear decay $1 - d/100$	0.0%	0.782	+0.329

10 Our environment and the settings we changed

Item	Paper	Ours
Hardware	A100 80GB, H200 140GB	Apple M3 Pro, 11 cores, 18 GB, CPU only
Python	not stated	3.13
PyTorch	not stated	2.13.0
PyTorch Geometric	not stated	2.8.0.post1
Total compute	50 GPU-hours pre-training	13 minutes evaluation

Settings we changed, and why.

Flag	Our value	Reason
<code>-num_workers</code>	0	<code>NormalizeData.__init__</code> builds a <code>defaultdict(lambda: ...)</code> , which is unpicklable, so spawn-based <code>DataLoaders</code> fail
<code>-batch_size</code> (BRACS)	8	1,888 nodes per graph on average, 15,496 at maximum; the default 2048 needs tens of gigabytes for the concatenated hidden states
<code>-batch_size</code> (BreakHis)	64	same reason, smaller graphs
<code>-group_regex</code>	see caption	the NuCLS <code>splits/</code> directory is absent from the repository; we grouped on <code>^(TCGA-[A-Z0-9]+)</code>
<code>-num_folds</code>	5	matches the dataset's documented 5-fold hospital protocol
<code>-tissue_filter</code>	Breast	no filter shipped for the PanNuke breast subset
<code>-label_map</code>	nucls_super	no 7→4 mapping shipped

Every value in this document was read from the released source at github.com/ogutsevda/graphist, from the paper and its appendices, or measured directly. Where a value is stated in only one of those places, the source is named.